Assignment_4
Classification

Antati Sravani (2023101013)
Keerthana Korlapati (2023101121)

October 2025

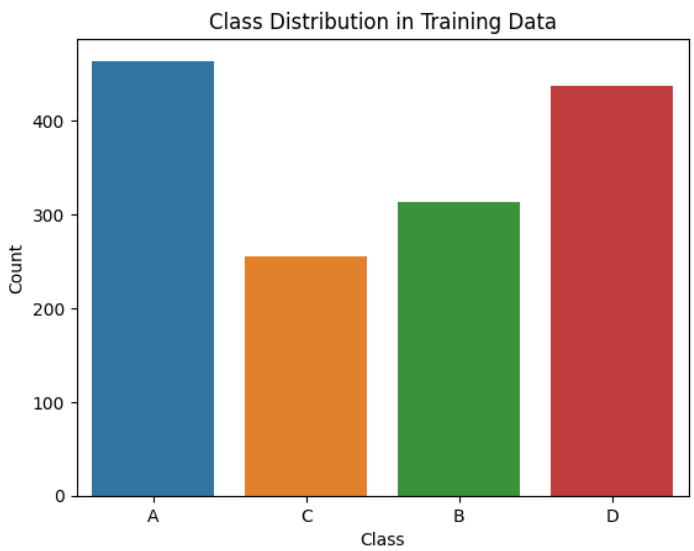
1 Effect of Class Imbalance in Multiclass Classification and Mitigation Strategies

In multiclass classification, the dataset contains multiple categories or labels that the model must learn to distinguish. However, when the number of samples per class is not evenly distributed, the dataset is said to have a **class imbalance problem**. This issue is very common in real-world datasets (for example, fraud detection, medical diagnosis, or customer segmentation). Class imbalance can severely affect a model's performance, particularly when the dataset is small.

1.1 Effect of Class Imbalance

Class imbalance causes the learning algorithm to become biased toward the majority classes, as they dominate the loss function during training. This leads to several specific problems:

- **Biased Predictions:** The model tends to predict majority classes more often, neglecting minority classes.
- **Poor Recall and Precision for Minority Classes:** The model may have high accuracy overall but fail to identify minority samples correctly (low recall).
- **Skewed Decision Boundaries:** The classifier adjusts its boundary closer to minority regions, reducing their separability.
- **Metric Misinterpretation:** Accuracy can be misleading — a model predicting only the majority class can still appear "accurate."
- **Worsened Effect on Small Datasets:** When data is already limited, minority classes have very few examples, preventing the model from learning their underlying patterns effectively.



1.2 Strategies to Mitigate Class Imbalance

Several techniques can be used to handle class imbalance and improve model fairness across all classes, especially for small datasets.

1.2.1 Data-Level Approaches (Resampling)

- **Oversampling Minority Classes:** Increases the representation of under-represented classes by replicating or synthetically generating new samples.
  Techniques: SMOTE (Synthetic Minority Over-sampling Technique), ADASYN.
- **Undersampling Majority Classes:** Reduces the number of samples from overrepresented classes to balance the dataset.
  Techniques: Random Undersampling, Cluster Centroid Sampling.
- **Hybrid Sampling:** Combines both over- and undersampling to balance data while maintaining diversity.

1.2.2 Algorithm-Level Approaches

- **Class Weighting:** Assign higher penalties to misclassified samples from minority classes. Most models (like SVM, RandomForest, LogisticRegression) support `class_weight='balanced'`.
- **Cost-Sensitive Learning:** Integrates the cost of misclassifying minority samples directly into the model's objective function.

1.2.3 Ensemble Methods

- **Bagging and Boosting Approaches:** Ensemble models like Random Forests or XGBoost can handle imbalance more robustly.
- **Boosting (e.g., AdaBoost, LightGBM):** Gives higher weights to misclassified samples, improving minority detection.

1.2.4 Evaluation Strategies

When data is imbalanced, accuracy is not a reliable metric. Instead, use the following evaluation measures:

- Precision, Recall, F1-Score (macro or weighted average)
- Confusion Matrix to see per-class performance
- ROC Curves and AUC per class
- Precision–Recall Curves, especially when imbalance is severe

2 Effect of Hyperparameters on Overfitting and Underfitting in Random Forest and SVM

Machine learning models like **Random Forest** and **Support Vector Machine (SVM)** rely heavily on their hyperparameters, which control model complexity and generalization capability. Improper tuning of these parameters can lead to **overfitting** (the model fits training data too well and performs poorly on unseen data) or **underfitting** (the model is too simple to capture underlying patterns).

2.1 Random Forest Classifier

Random Forest is an ensemble of decision trees, where hyperparameters determine the depth, size, and randomness of individual trees.

2.1.1 Hyperparameters Leading to Overfitting

- **max_depth (Too large or None):** When trees are allowed to grow deep without restriction, they capture noise and specific details of training data, leading to overfitting. Each tree becomes highly specialized, reducing the generalization power.
- **min_samples_split or min_samples_leaf (Too small):** When very few samples are required to split a node or form a leaf, the model creates many small, highly specific branches. This leads to complex decision boundaries that fail on new data.

- **n_estimators (Too high without regularization):** While more trees generally improve performance, an excessively large number of trees can cause the model to memorize the training data if other regularization parameters are weak.

2.1.2 Hyperparameters Leading to Underfitting

- **max_depth (Too small):** If trees are too shallow, the model cannot capture complex relationships and produces overly simple predictions.
- **n_estimators (Too few):** A small number of trees reduces ensemble diversity and fails to represent data variability.
- **max_features (Too low):** If too few features are considered at each split, important attributes may be ignored, reducing the model's predictive strength.

2.2 Support Vector Machine (SVM) Classifier

SVM uses a kernel function to map data into higher dimensions and find an optimal separating hyperplane. Its sensitivity to hyperparameters like C, *kernel type*, and *gamma* determines model complexity.

2.2.1 Hyperparameters Leading to Overfitting

- **C (Too large):** A large C forces the SVM to classify all training points correctly, creating a very tight margin. This increases model variance and causes overfitting, especially in noisy datasets.
- **gamma (Too high, especially with RBF kernel):** A high gamma makes the decision boundary highly sensitive to individual data points. The model creates complex, wiggly boundaries that fit training data perfectly but generalize poorly.

2.2.2 Hyperparameters Leading to Underfitting

- **C (Too small):** A very small C value makes the margin too wide and tolerant of misclassifications. The model fails to capture data structure, resulting in poor performance on both training and test sets.
- **gamma (Too low):** A small gamma produces smoother, simpler boundaries, causing the model to overlook important variations in data.
- **kernel (Unsuitable choice):** Using a linear kernel for inherently non-linear data can result in underfitting, as the model cannot capture complex relationship.

3 Plots

3.1 OVO Plots

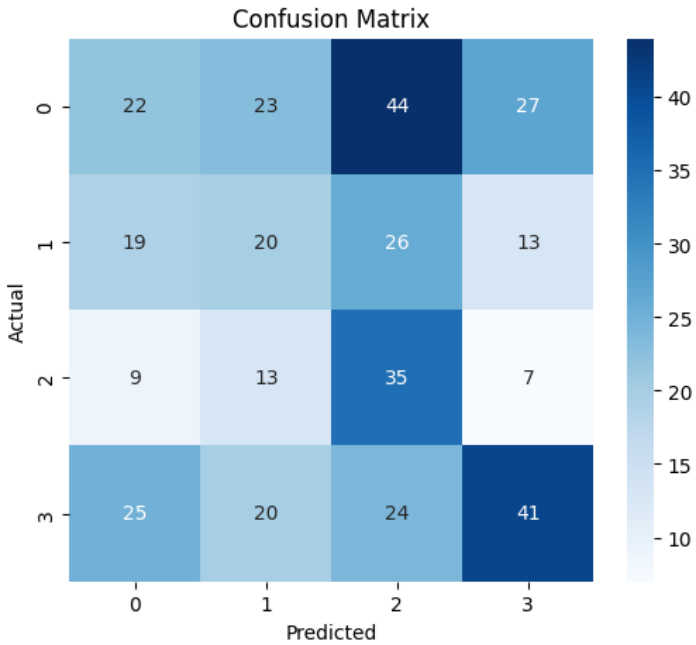


Figure 1: OVO Confusion Matrix

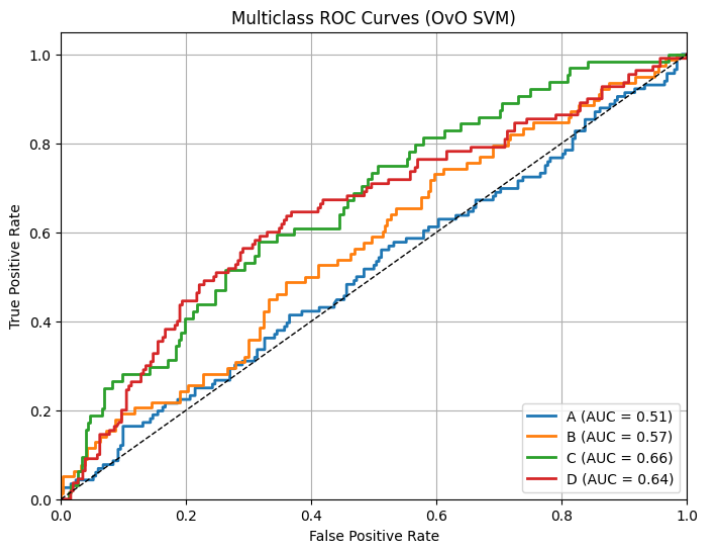


Figure 2: OVO ROC Curve

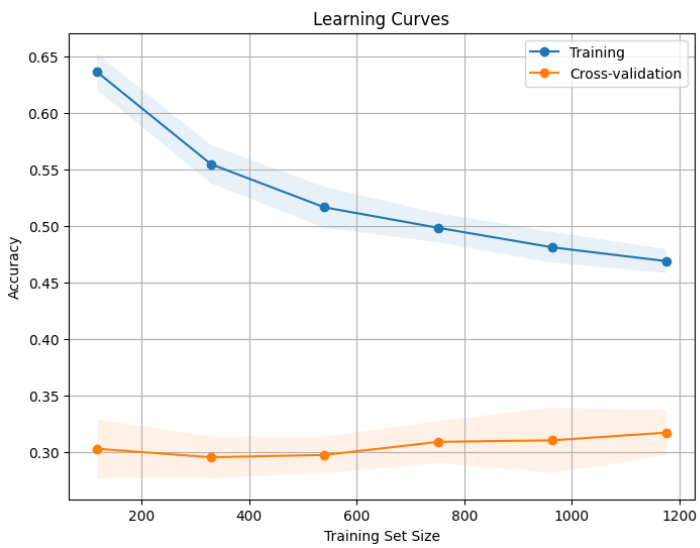


Figure 3: OVO Loss Curve

Table 1: Model Evaluation Summary (OVO)

Class	Precision	Recall	F1-score	Support
A	0.29	0.19	0.23	116
B	0.26	0.26	0.26	78
C	0.27	0.55	0.36	64
D	0.47	0.37	0.41	110
Accuracy		0.32 (368 samples)		
Macro Avg	0.32	0.34	0.32	368
Weighted Avg	0.33	0.32	0.31	368

3.2 OVA Plots

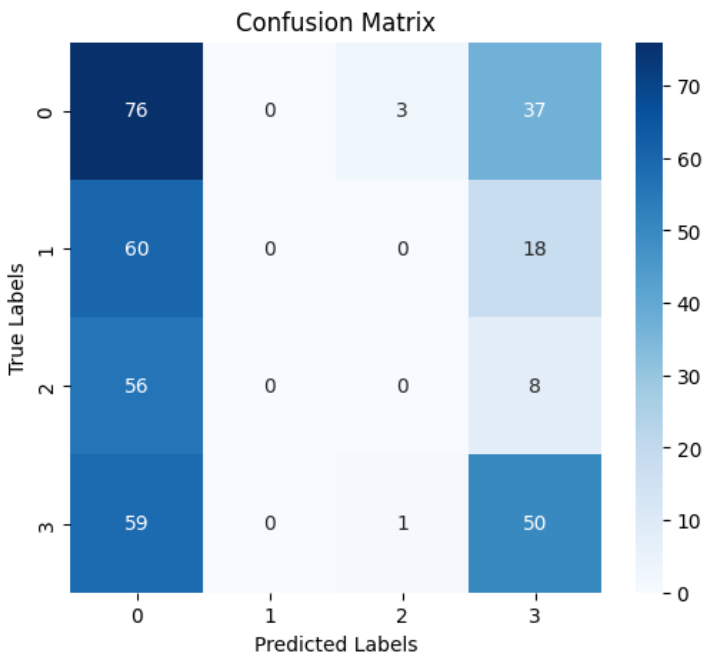


Figure 4: OVA Confusion Matrix

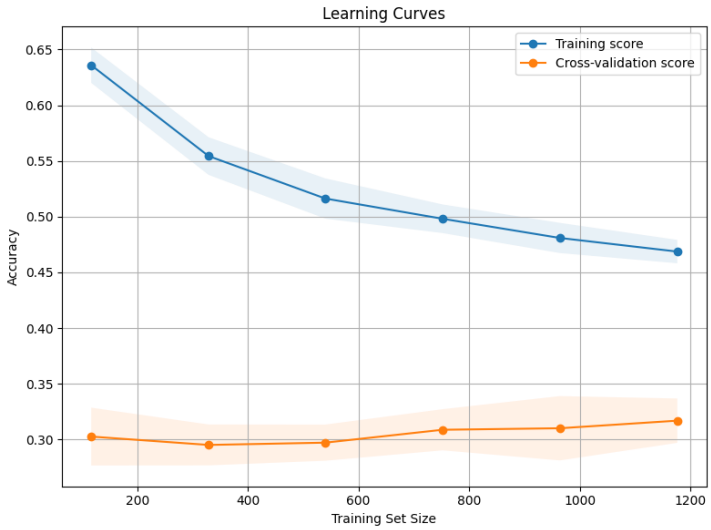


Figure 5: OVA Loss Curve

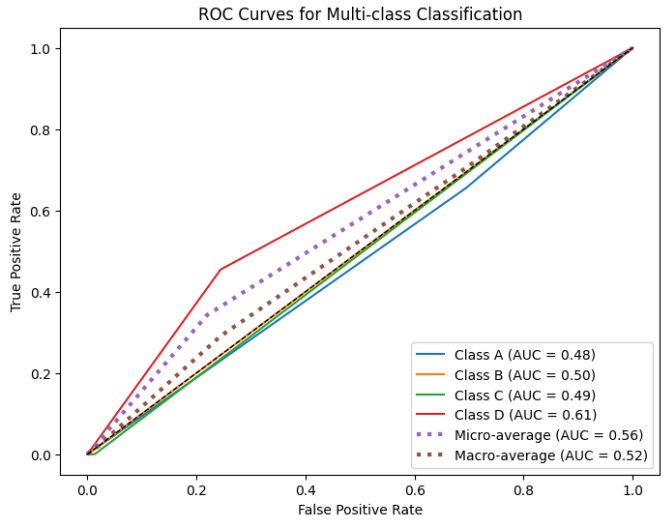


Figure 6: OVA ROC Curve

Table 2: Classification Report (OVA)

Class	Precision	Recall	F1-score	Support
A	0.30	0.66	0.41	116
B	0.00	0.00	0.00	78
C	0.00	0.00	0.00	64
D	0.44	0.45	0.45	110
Accuracy		0.34 (368 samples)		
Macro Avg	0.19	0.28	0.22	368
Weighted Avg	0.23	0.34	0.26	368

3.3 Random Forest Plots

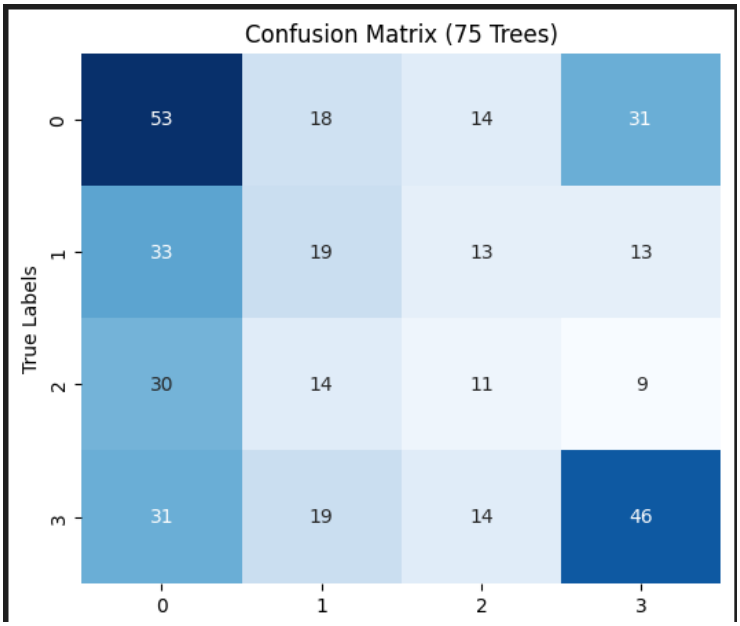


Figure 7: Random Forest Confusion Matrix

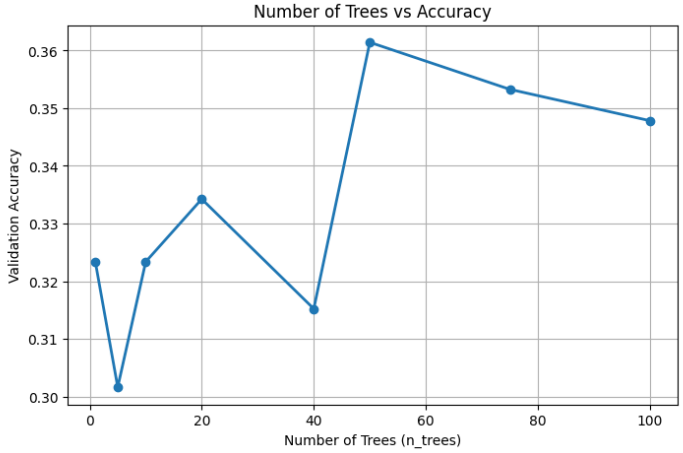


Figure 8: Random Forest Feature Importance or Learning Curve

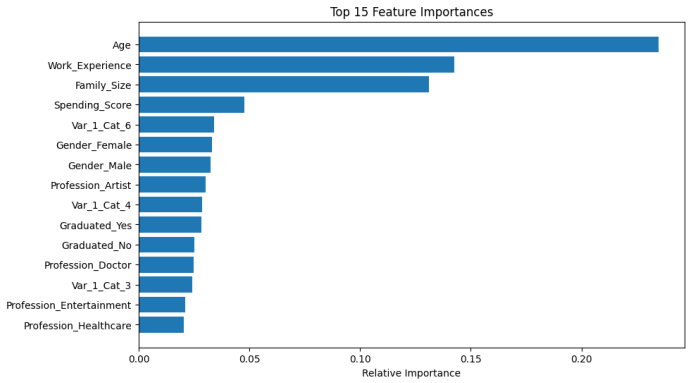


Table 3: Classification Report (Random Forest)

Class	**Precision**	**Recall**	**F1-score**	**Support**
A	0.36	0.46	0.40	116
B	0.27	0.24	0.26	78
C	0.21	0.17	0.19	64
D	0.46	0.42	0.44	110
Accuracy		0.35 (368 samples)		
Macro Avg	0.33	0.32	0.32	368
Weighted Avg	0.35	0.35	0.35	368

4 Comparison of One-vs-All (OvA) and One-vs-One (OvO) Classification Results

Table 4: Comparison of OvA and OvO Classification Performance

Metric	**OvA**	**OvO**
Accuracy	34%	32.07%
Macro Precision	0.19	0.32
Macro Recall	0.28	0.34
Macro F1-score	0.22	0.32
Weighted F1-score	0.26	0.31

4.1 Analysis

4.1.1 (a) One-vs-All (OvA)

Strengths:

- OvA achieved a slightly higher accuracy (34%) compared to OvO (32%), indicating it classified more samples correctly overall.

Weaknesses:

- The precision and recall values for most classes (especially B and C) are very low or zero, showing that the OvA model struggled to separate these minority or overlapping classes.
- OvA classifiers often face class imbalance issues, as each binary model compares one class against all others, leading to biased decision boundaries.

4.1.2 (b) One-vs-One (OvO)

Strengths:

- OvO achieved more balanced performance across classes — the macro precision, recall, and F1-scores are all higher (around 0.32–0.34).
- The model handles class overlap better since it focuses on pairwise distinctions rather than a single class versus all others.
- Classes C and D show notably better F1-scores (0.36 and 0.41), reflecting improved per-class discrimination.

Weaknesses:

- Slightly lower overall accuracy (32.07%), likely due to noisy decisions when combining multiple binary classifiers through voting.
- Higher computational cost since it trains $\frac{k(k-1)}{2}$ classifiers (for 4 classes $\rightarrow$ 6 classifiers).

4.2 Interpretation

- OvA produces higher overall accuracy but performs poorly on minority classes, indicating class bias.
- OvO provides more stable and balanced per-class performance, even though overall accuracy is marginally lower.
- From a macro-level performance perspective (macro precision, recall, and F1), OvO outperforms OvA because it treats all classes more equally rather than favoring dominant ones.